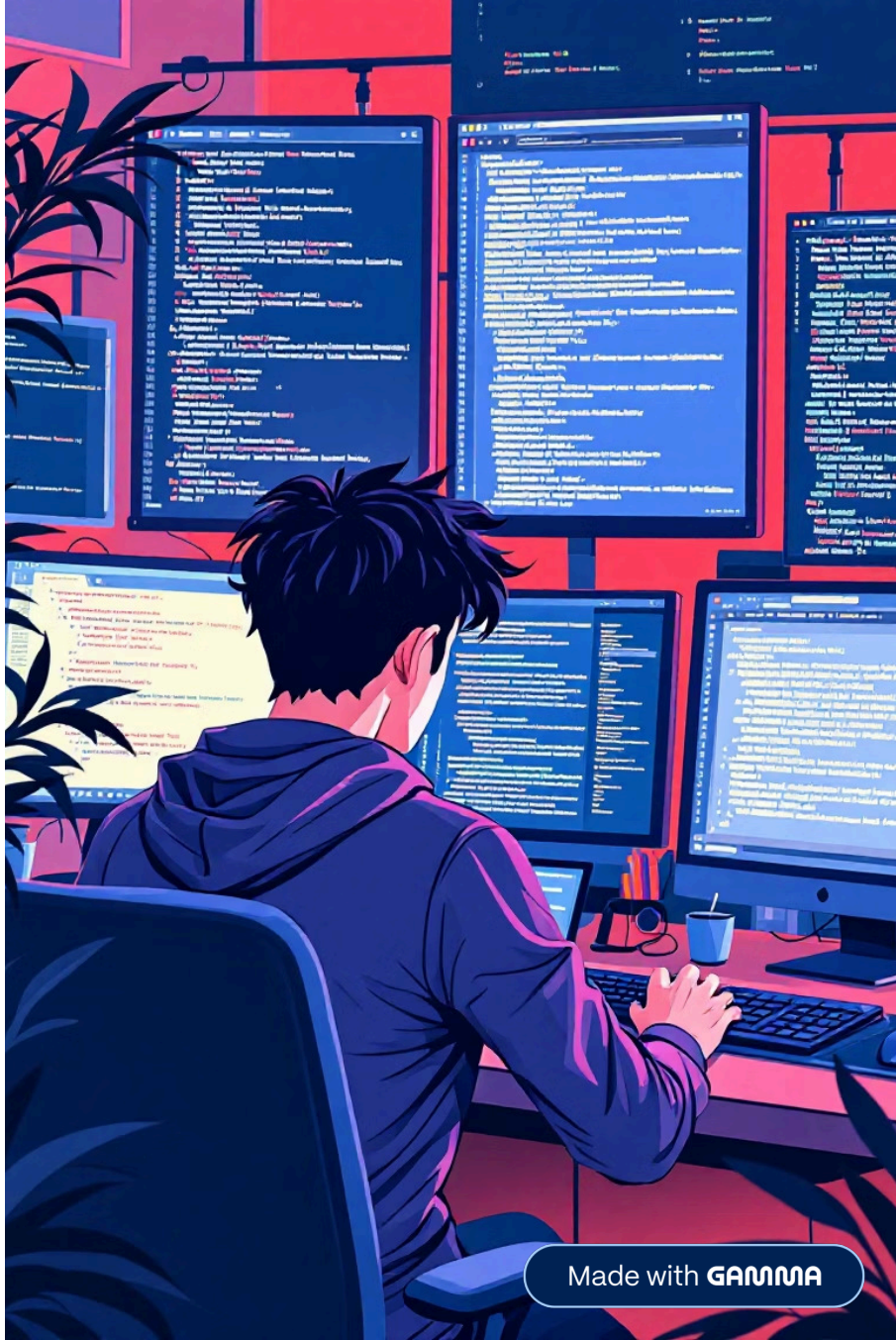
Git & GitHub Mastery

From absolute beginner to advanced practitioner — your complete guide to version control and collaborative development with Git and GitHub.



Made with GAMMA

Why Version Control Matters

Version control systems track changes to your code over time, enabling you to collaborate safely, recover from mistakes, and maintain a complete history of your project's evolution.

Track Changes

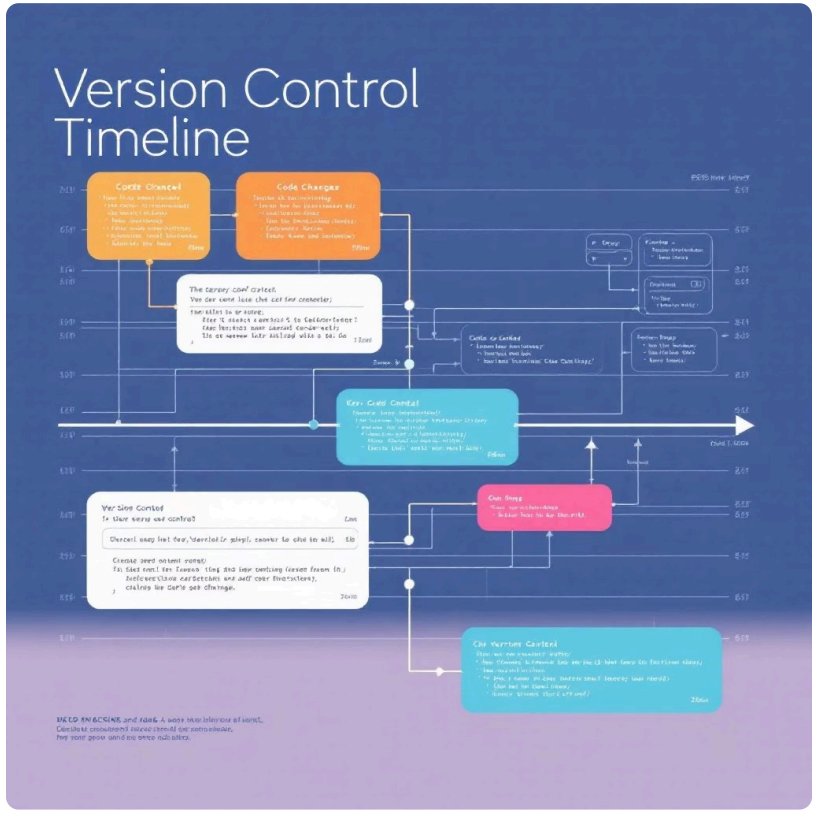
Every modification is recorded with timestamps and author details

Safe Collaboration

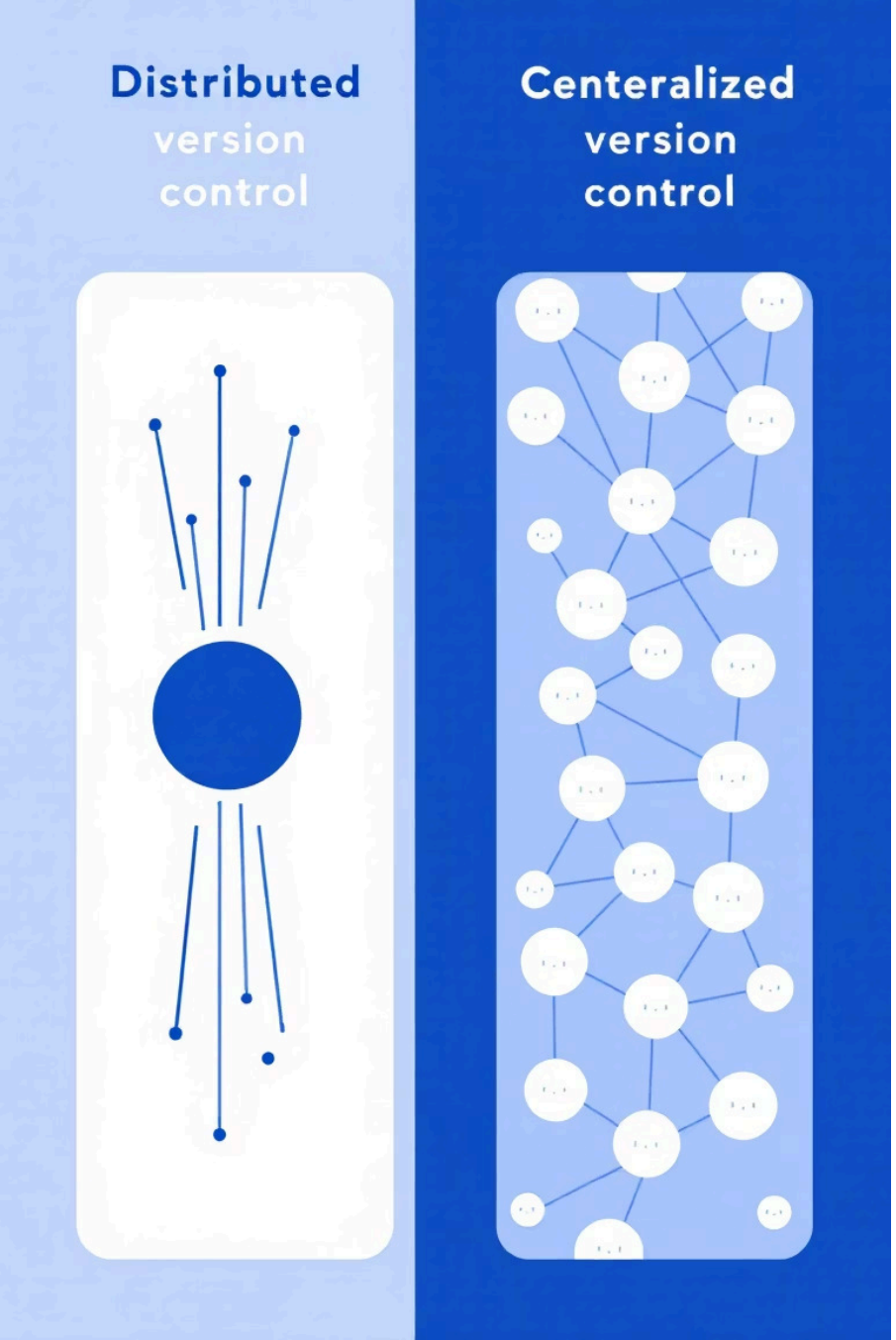
Multiple developers can work simultaneously without conflicts

Error Recovery

Easily revert to previous working versions when bugs arise



Made with GAMMA



Git vs GitHub: Understanding the Difference

git

Git

Distributed version control system that runs locally on your machine. Tracks changes, manages branches, and handles merging.

GitHub

Cloud-based hosting service for Git repositories. Adds collaboration features, issue tracking, and deployment tools.

Think of Git as the engine and GitHub as the garage where you park and share your projects with others.

Made with GAMMA

Getting Started: Installation & Setup

01

Install Git

Download from git-scm.com for Windows/Mac, or use package managers like apt/brew for Linux/Mac systems.

02

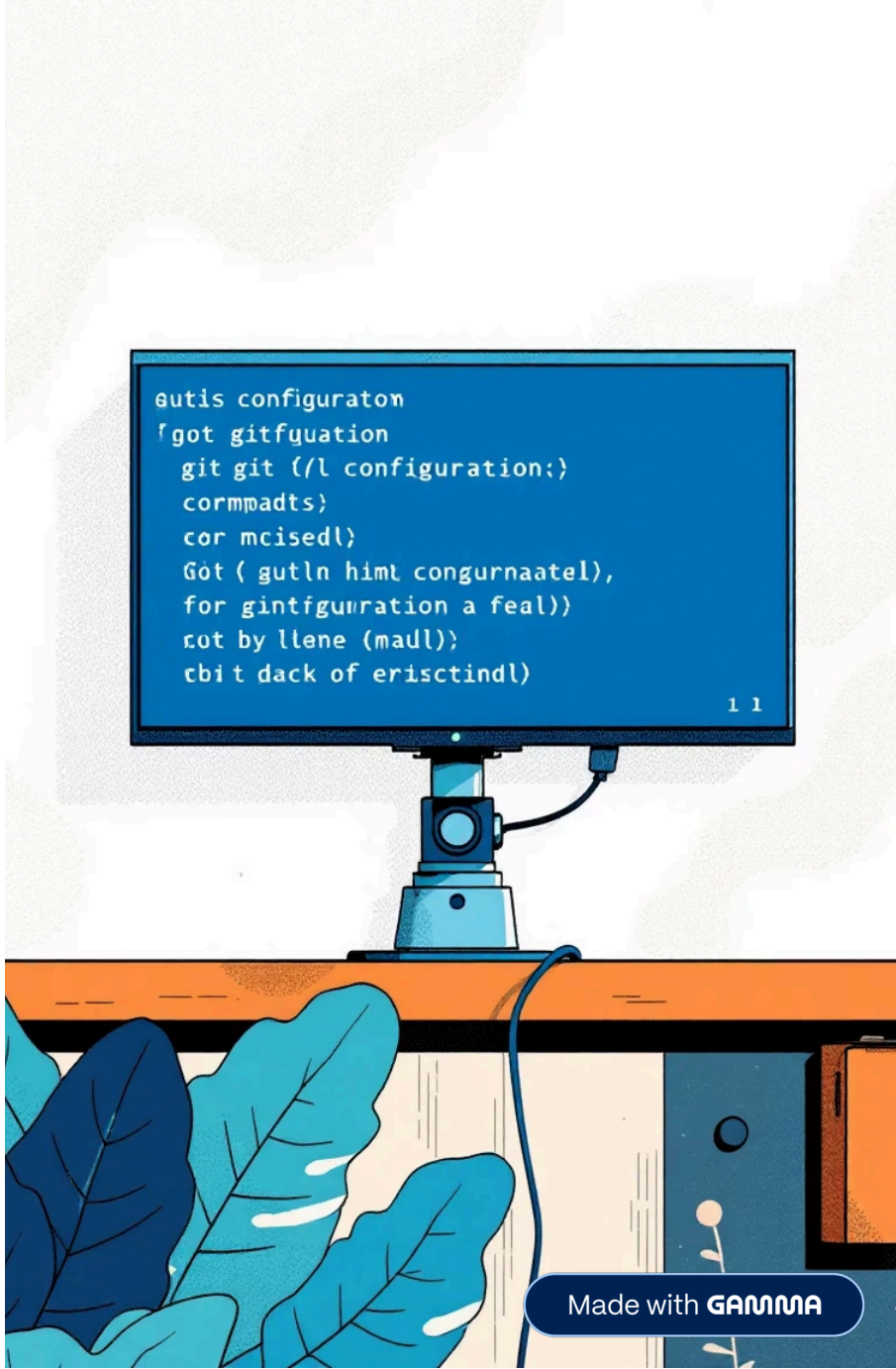
Configure Identity

```
git config --global user.name "Your Name"
git config --global user.email "you@example.com"
```

03

Verify Installation

```
git --version
git config --list
```



Made with GAMMA

Git Fundamentals: Your First Repository

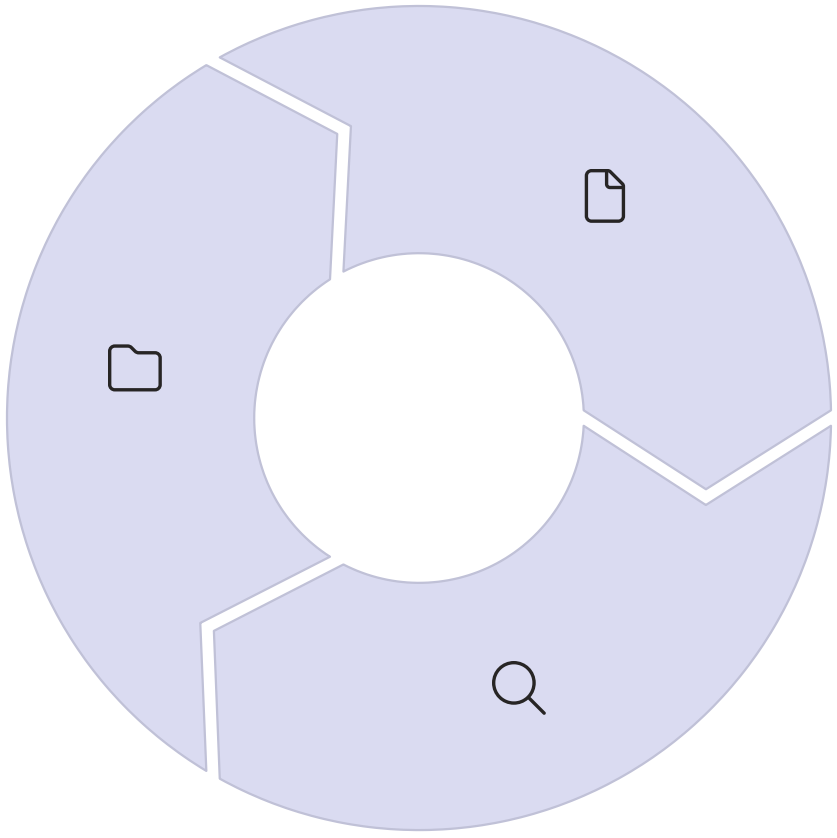
Creating a New Repository

```
mkdir my-project
cd my-project
git init
```

Cloning an Existing Repository

```
git clone https://github.com/user/repo.git
cd repo
```

The .git folder contains all version control data. Never delete it manually!



Working Directory

Your actual files

Staging Area

Files ready to commit

Repository

Committed snapshots

Made with GAMMA

Core Git Workflow: Stage, Commit, Repeat

Check Status

```
git status
```

See which files are modified, staged, or untracked

Stage Changes

```
git add filename.txt
git add .  # stage all changes
```

Prepare files for the next commit

Commit Changes

```
git commit -m "Add user authentication feature"
```

Create a permanent snapshot with descriptive message

View History

```
git log --oneline
```

Review your commit history and track progress

Made with GAMMA

Mastering Branches & Merging

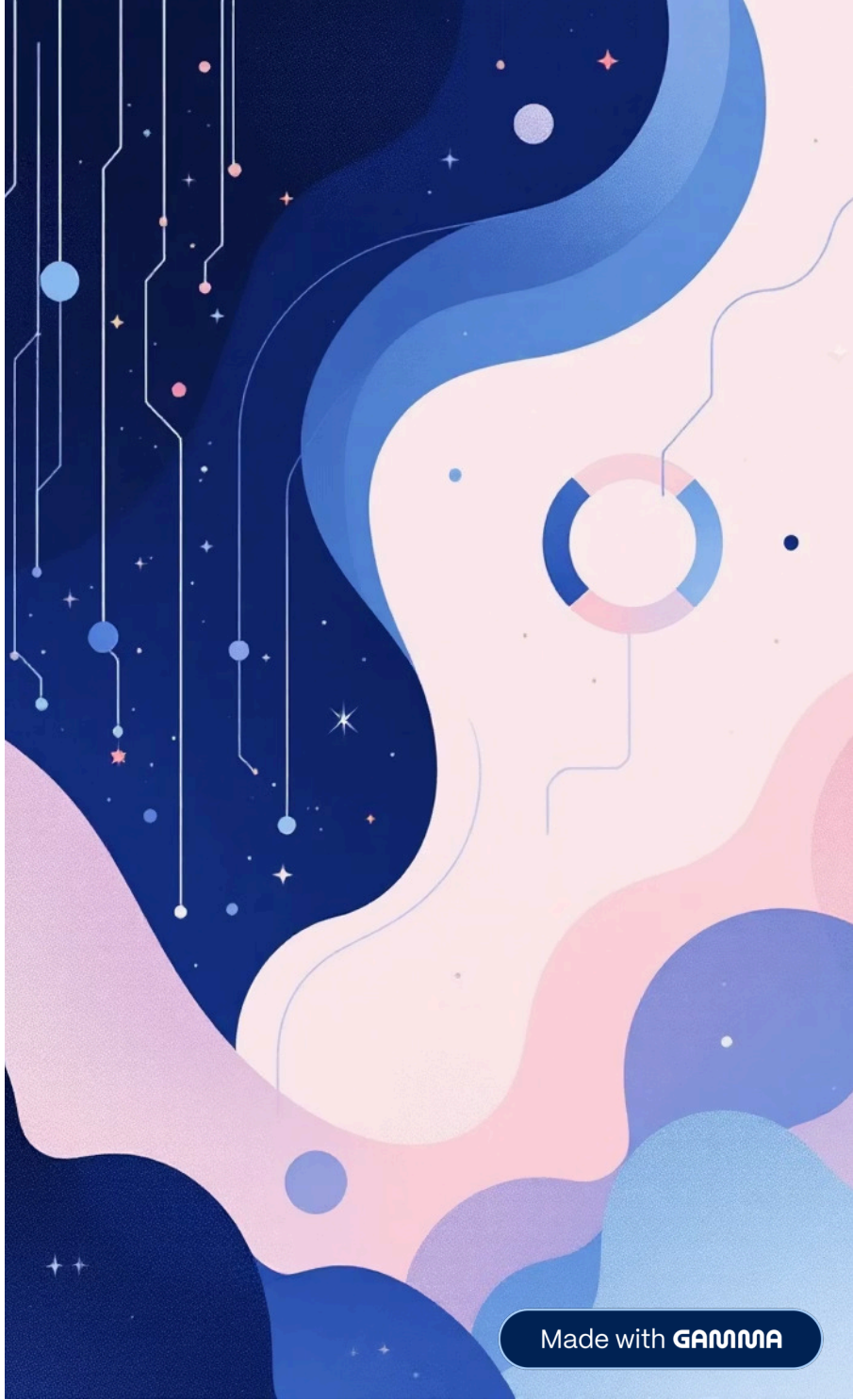
Branch Operations

```
# Create and switch to new branch
git checkout -b feature-login

# List all branches
git branch -a

# Switch branches
git checkout main
```

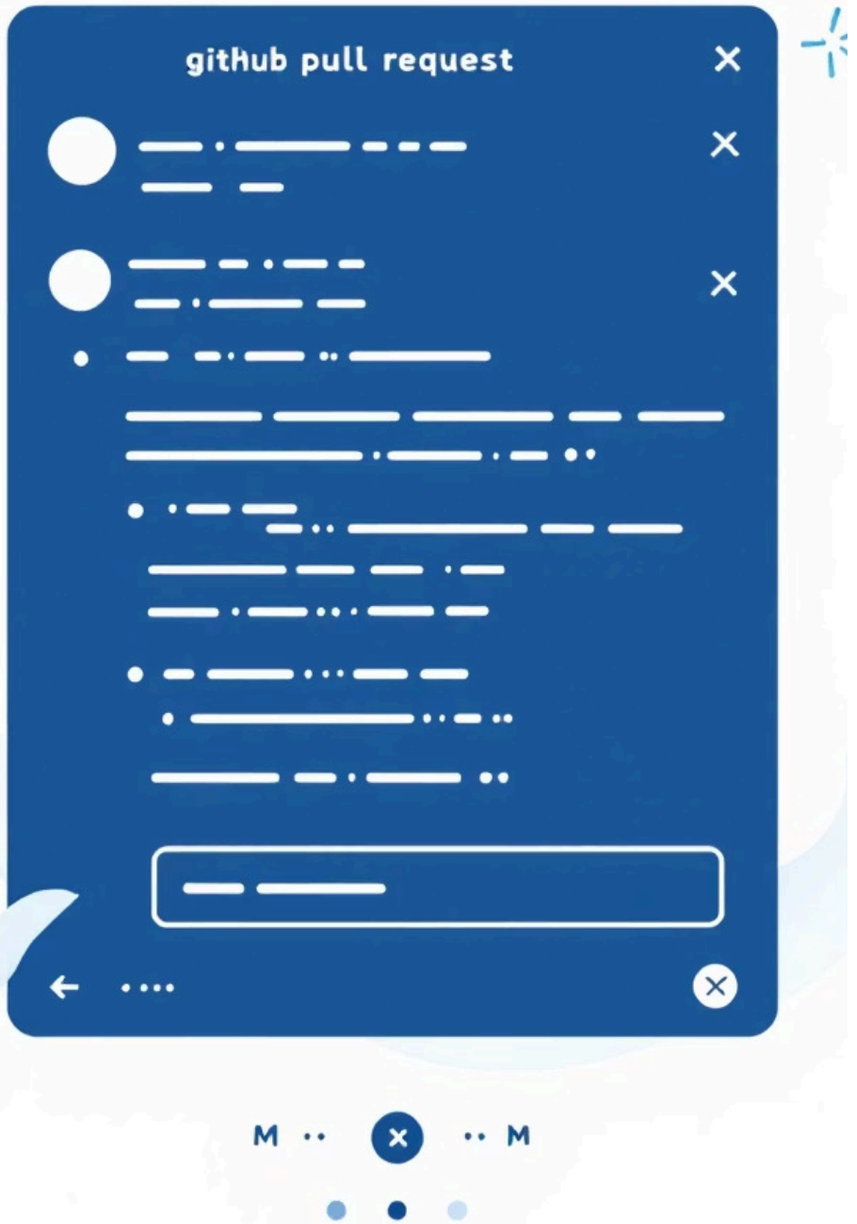
Merging Strategies

```
# Merge feature branch
git checkout main
git merge feature-login

# Delete merged branch
git branch -d feature-login
```

⚠ Always pull the latest changes before creating new branches to avoid conflicts later.



Made with GAMMA



GitHub Collaboration Workflow

1. **Fork Repository**

   Create your own copy of the project on GitHub

2. **Clone & Branch**

   ```
   git clone your-fork-url
   git checkout -b new-feature
   ```

3. **Make Changes**

   Code, test, and commit your improvements

4. **Push & Pull Request**

   ```
   git push origin new-feature
   ```

   Submit changes for review

Pull requests enable code review, discussion, and controlled integration of changes into the main project.

Made with GAMMA

Advanced Git Techniques

Stashing

```
git stash
git stash pop
```

Temporarily save uncommitted changes when switching contexts

Rebasing

```
git rebase main
```

Replay commits on top of another branch for cleaner history

Cherry-picking

```
git cherry-pick abc123
```

Apply specific commits from one branch to another

Tagging

```
git tag v1.0.0
git push origin --tags
```

Mark specific points in history as important releases

Made with GAMMA

Essential Git & GitHub Cheatsheet

Daily Commands

```
git status        # Check repository status
git add .         # Stage all changes
git commit -m "msg" # Commit with message
git push          # Upload to remote
git pull          # Download updates
git log --oneline   # View commit history
```

Branch Management

```
git branch        # List branches
git checkout -b new # Create & switch
git merge branch    # Merge into current
git branch -d old   # Delete branch
```

Emergency Commands

```
git reset HEAD~1    # Undo last commit
git checkout file   # Discard changes
git revert abc123   # Safe undo commit
git stash           # Save work temporarily
```

Remote Operations

```
git clone url       # Copy repository
git remote -v       # Show remotes
git fetch           # Download updates
git push origin br  # Push specific branch
```

Remember: Commit often, pull before pushing, and write meaningful commit messages. Happy coding!

Made with GAMMA